UFUG 2602
基本排序与链表

Beiyan Cao, Tengfei Chang

CES and IoT Thrusts
香港科技大学（广州）

大纲

- 基本排序
  - **冒泡排序**
  - 选择排序
  - 计数排序
- 链表
  - 单向链表
  - 双向链表
  - 链式序列与数组序列

排序的一个核心思想

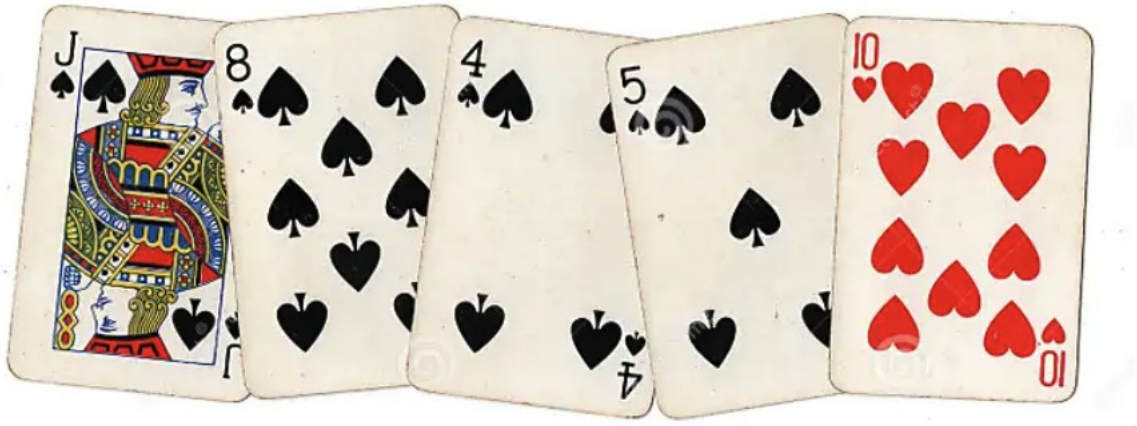
- 始终把数组看作两部分：已排序部分在前，未排序部分在后。
  - 最初已排序部分为空，整个数组都未排序。
  - 每次迭代再加入一个数以扩大已排序部分，并相应缩小未排序部分。

- 插入排序是该思想的实例。本讲再介绍两种基于同一思想的算法。

冒泡排序 – 示例



- 设想你在和朋友打扑克。
- 每人手上有一副牌，任务是把牌从低到高排好。
- 但你只能查看并交换相邻的两张牌。



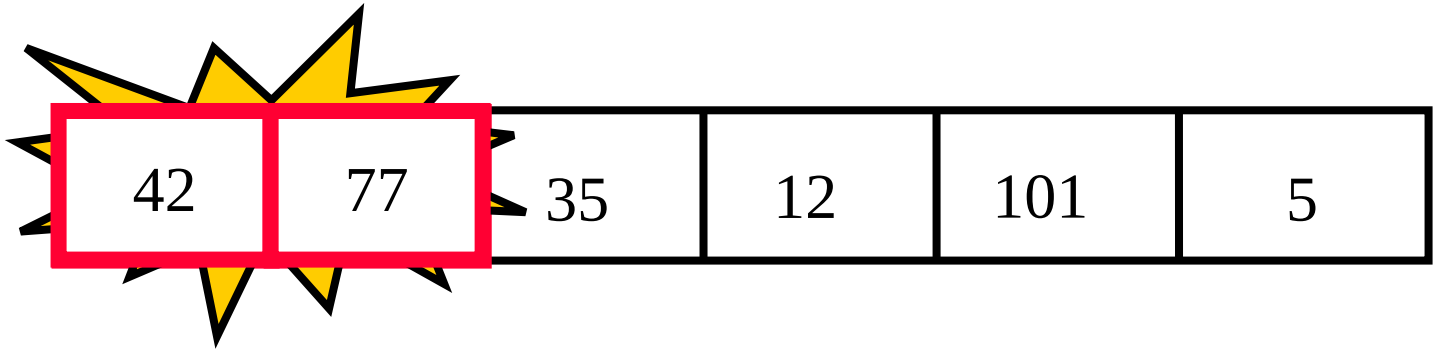
将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾

77	42	35	12	101	5

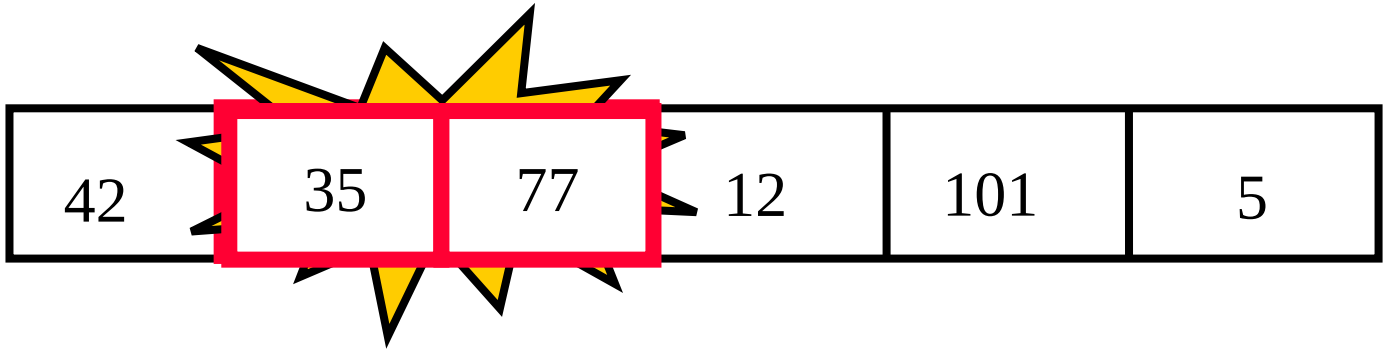
将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾



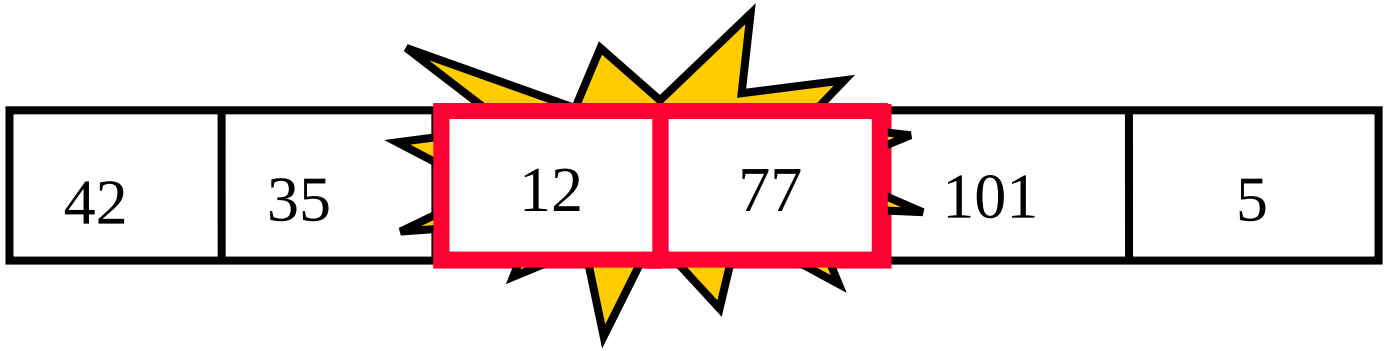
将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾



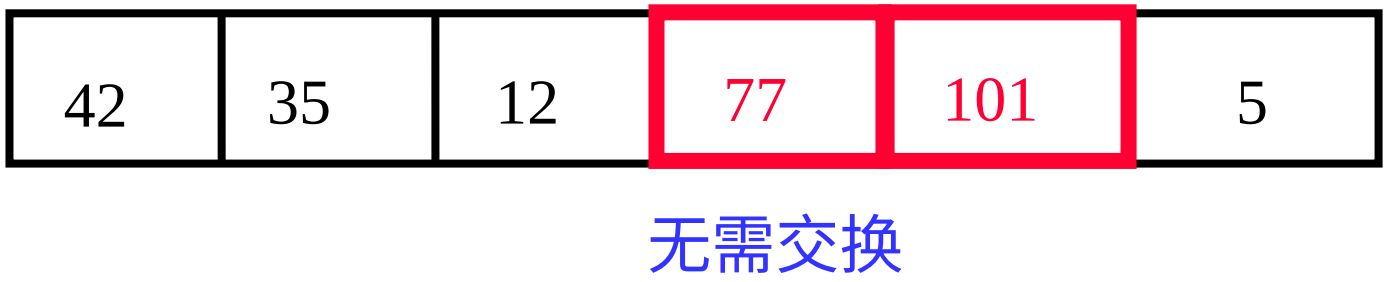
将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾



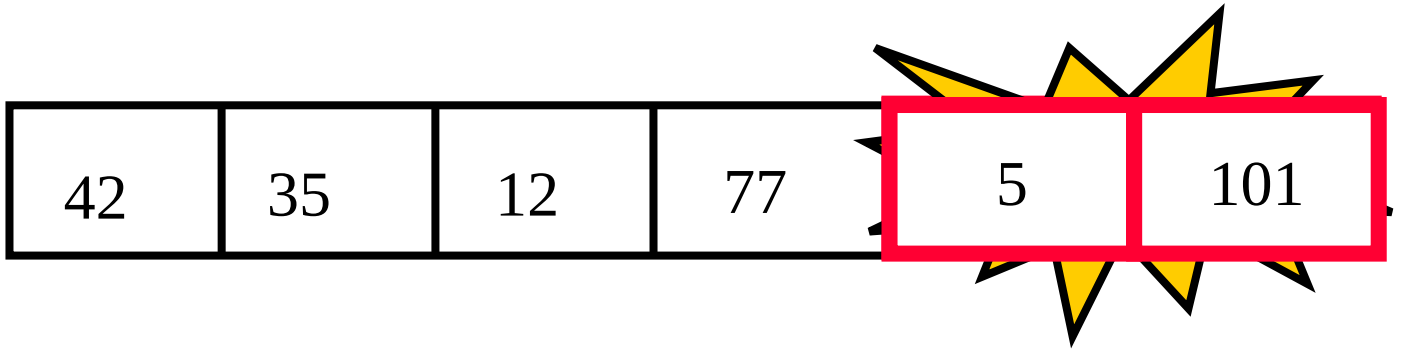
将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾



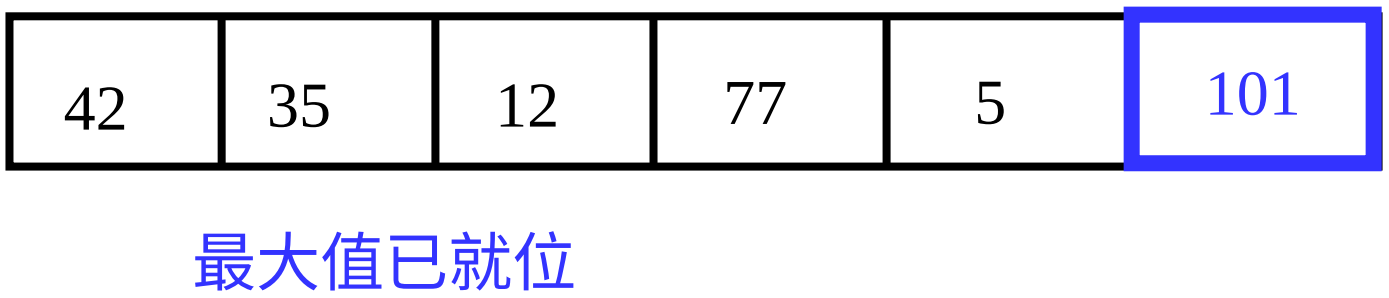
将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾



将最大元素“上浮”

- 遍历一组元素
- 从前往后扫描
- 通过两两比较与交换，把最大值“冒泡”到末尾



“上浮”算法

算法： BUBBLE-UP(A)

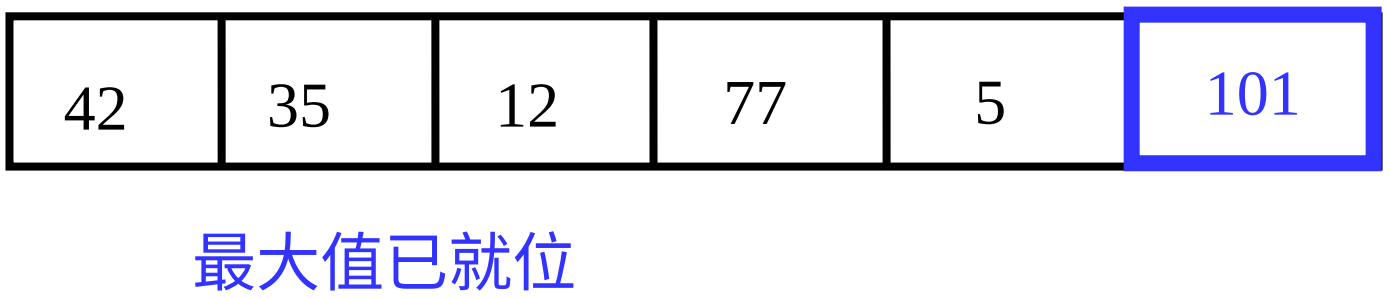
for j ← 1 **to n – 1**

if A[j] < A[j -1]

然后交换 A[j] ↔ A[j-1]

关注要点

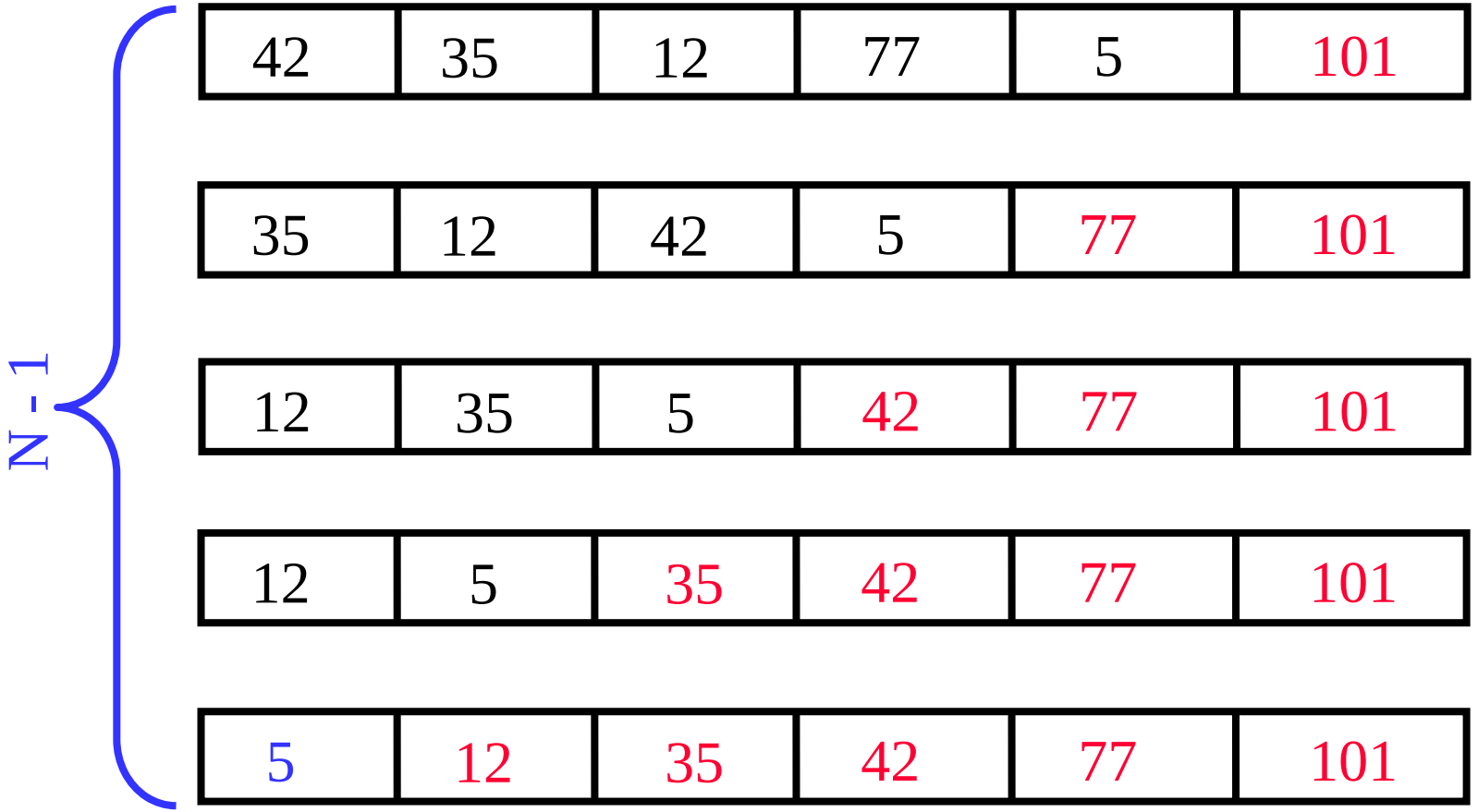
- **注意此时只有最大值已正确就位**
- **其余元素仍未排好**
- **因此需要重复该过程**



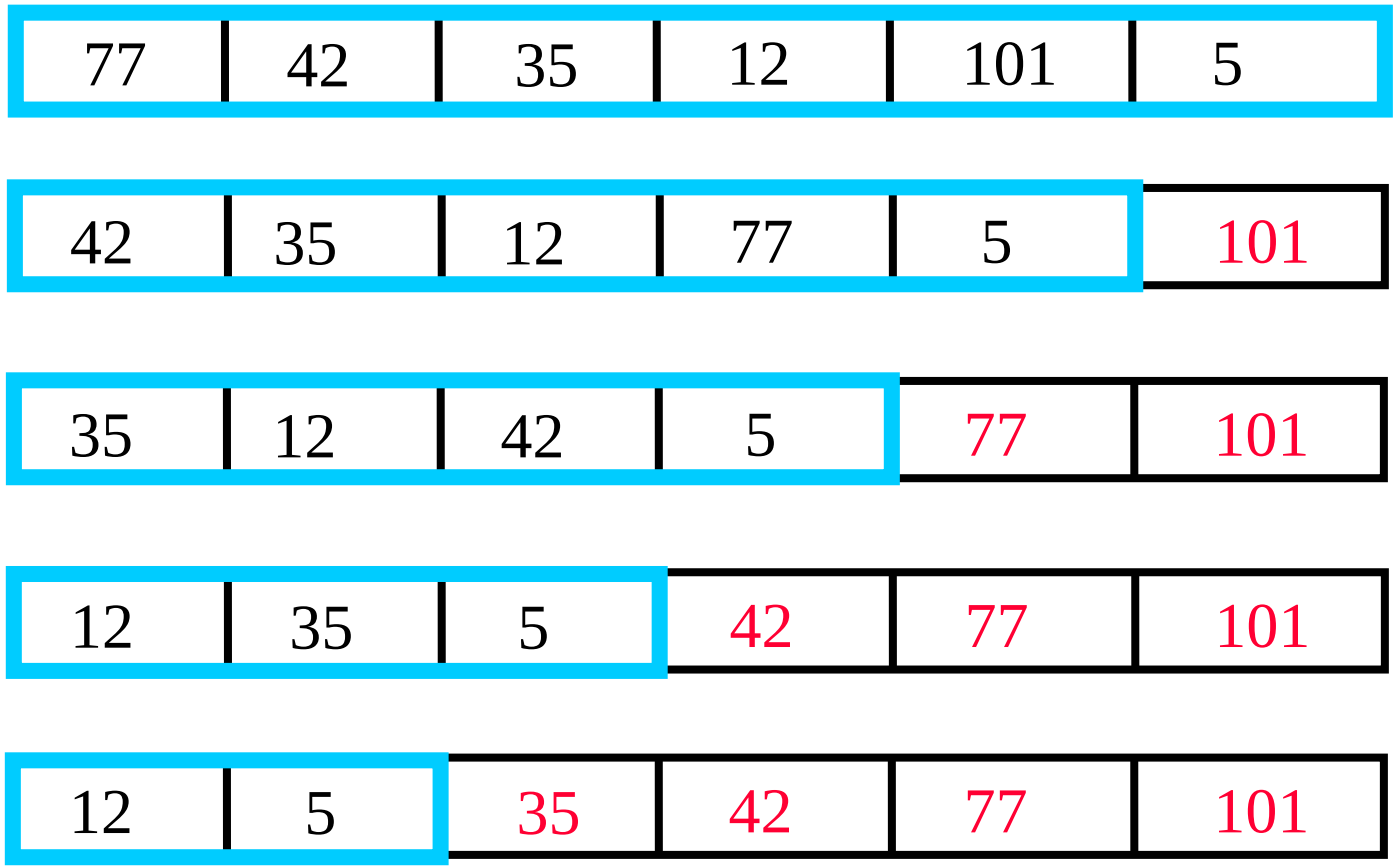
“上浮”需要重复多少次?

- 如果我们有 N 个元素…
- 并且如果我们每次冒泡一个元素，就把它放到正确的位置…
- 然后我们把“上浮”过程重复 $N-1$ 次。
- 这保证我们能正确放置全部 N 个元素。

让所有元素都“上浮”


N - 1
42 35 12 77 5 101
35 12 42 5 77 101
12 35 5 42 77 101
12 5 35 42 77 101
5 12 35 42 77 101

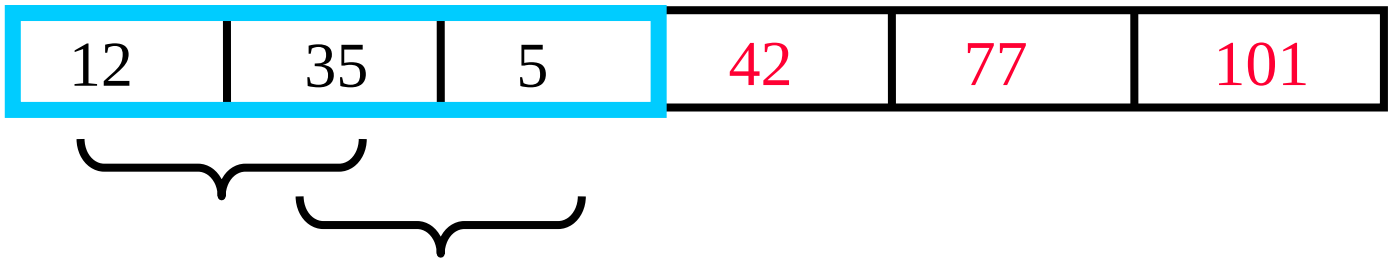
减少比较次数


77 42 35 12 101 5
42 35 12 77 5 101
35 12 42 5 77 101
12 35 5 42 77 101
12 5 35 42 77 101

减少比较次数

- **第 i 次“上浮”只需做 N-i 次比较。**

- **例如:**
- **这是第 4 次“上浮”**
- **N 为 6**
- **因此只需做 2 次比较**



已经有序的序列?

- **若数组 A 已经有序呢?**
- **若只有少数元素错位，几次“上浮”后就已有序呢?**

- **如何检测并提前终止?**

5	12	35	42	77	101

已经有序的序列?

- **可用布尔变量判断本次“上浮”是否发生过交换。**
- **若未发生交换，则序列已经有序！**
- **该布尔“标志”需在每次“上浮”后复位。**

综合起来

```python
# Optimized Python program for implementation of Bubble Sort
def bubbleSort(arr):
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        swapped = False

        # Last i elements are already in place
        for j in range(0, n-i-1):

            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if (swapped == False):
            break
```

https://www.geeksforgeeks.org/bubble-sort-algorithm/

- **共有 *n* 次迭代，每次迭代的运行时间为 *O(n)***
- **最坏情况下的运行时间为 *O(n^2)***

练习 1

- 给定数组：
  - [8, 3, 6, 1, 5]
- 将该数组排好序需要多少轮**上浮**？
- 写出**每一轮完整上浮**后的数组。
- 算法在哪一轮可以提前终止？
- 每一轮各发生多少次交换？

大纲

- 基本排序
  - 冒泡排序
  - **选择排序**
  - 计数排序
- 链表
  - 单向链表
  - 双向链表
  - 链式序列与数组序列

选择排序 – 示例



设想你在图书馆按高度升序整理书架上的书。

为此，你决定采用如下方法：

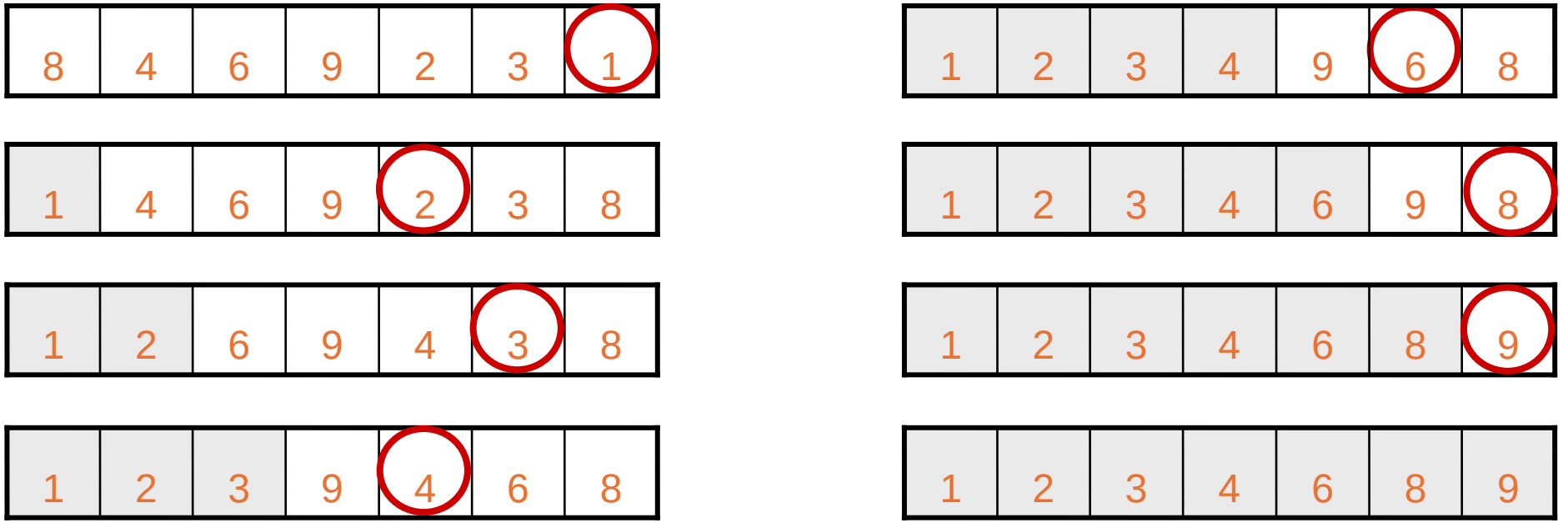
1. 先扫遍整架，找出最矮的书。
2. 把最矮的书放到第一个位置。
3. 移到下一位置，重复该过程：扫描剩余的书，找出下一本最矮的，放到下一位置。
4. 重复直至所有书按高度升序排好。



选择排序

- **思路：**
  - 找出数组中的最小元素
  - 与第一个位置的元素交换
  - 找出次小元素，与第二个位置的元素交换
  - 继续直到数组有序

选择排序 - 示例


8 4 6 9 2 3 1
1 4 6 9 2 3 8
1 2 6 9 4 3 8
1 2 3 9 4 6 8
1 2 3 4 9 6 8
1 2 3 4 6 9 8
1 2 3 4 6 8 9
1 2 3 4 6 8 9

选择排序 - 算法

算法： SELECTION-SORT(A)

```
n ← length[A]
for i ← 0 to n - 2
        do smallest ← i
           for j ← i + 1 to n - 1
                  do if A[j] < A[smallest]
                        then smallest ← j
交换 A[i] ↔ A[smallest]
```



- **共有 $O(n)$ 次迭代，每次迭代在 $O(n)$ 时间内完成。**
- **总运行时间为 $O(n^2)$**

递归选择排序

- 用递归重新定义迭代版选择排序算法

- **recursive_sort(A, 0, n-1):**
  1. 在 $O(n)$ 时间内找到数组中的最小元素
  2. 将它与第一个位置上的元素交换
  3. 我们现在得到一个子问题：
     - 通过调用 recursive_sort(A, 1, n-1) 对后面 n – 1 个元素排序

递归选择排序的分析

- 设 $f(n)$ 为问题规模为 n 时选择排序的最大运行时间。
- 已知： $f(1) = O(1)$

- 对于 $n \geq 2$，有：

$f(n) \leq O(n) + f(n-1)$

- 其中 $O(n)$ 表示第 1、2 步的代价，$f(n-1)$ 是第 3 步的代价。

递归选择排序的分析

- 因此只需求解该递推式（c_1, c_2 为常数）：
  - $f(1) = c_1$
  - $f(n) \le c_2 n + f(n-1)$
- 使用展开法，我们得到：
- $f(n) \le c_2 n + f(n-1)$
  - $\le c_2 n + c_2(n-1) + f(n-2)$
  - $\le c_2 n + c_2(n-1) + c_2(n-2) + f(n-3)$
  - $\le c_2 n + c_2(n-1) + \ldots + c_2 \cdot 2 + f(1)$
  - $\le c_2 n(n+1)/2 + c_1 = O(n^2)$.
- 因此我们得出：选择排序的最坏情况运行时间为 $O(n^2)$ 。

练习 2

- 对以下序列执行选择排序：
  - [7, 2, 9, 4, 1]
- 对每一轮：
- 找出未排序部分中的**最小元素**。

测验

- 哪种交换次数更少?
  - 冒泡排序
  - 选择排序

- 若采用提前终止，对于已经有序的数组，哪种算法的比较次数可能更少?

大纲

- 基本排序
  - 冒泡排序
  - 选择排序
  - **计数排序**
- 链表
  - 单向链表
  - 双向链表
  - 链式序列与数组序列

一种简单的排序算法?

考虑数组 A，其中有 n 个整数。每个整数都在 $[1, U]$ 范围内，其中 $U \geq n$。例如：[2,3,8,7,1,2,2,2,7,3,9,8,2,1,4,2,4,6,9,2]。

一种简单直接的方法把这些数字排好序：

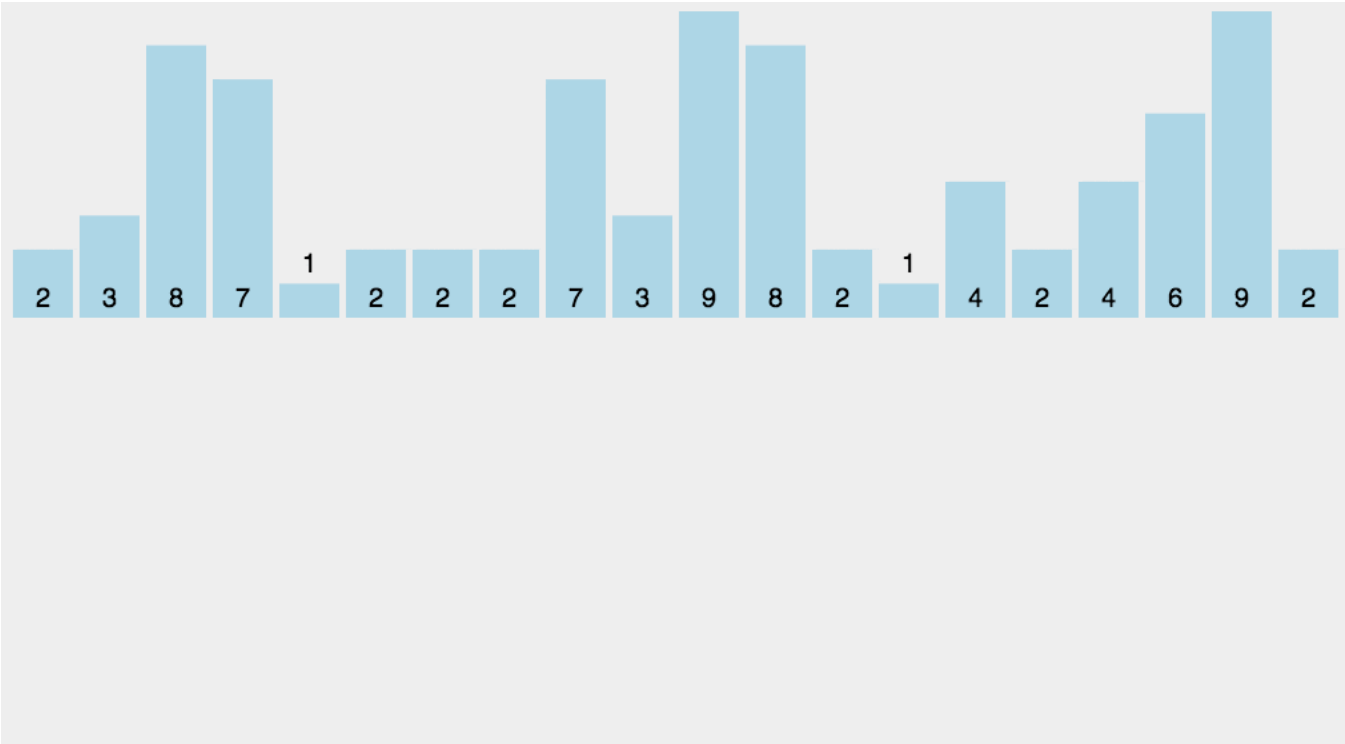
1. **创建计数数组 B，长度为 U。将 B 的所有单元初始化为 0。**
2. **统计出现次数：对每个 $i \in [0,n-1]$，将 $B[A[i]]$ 加 1。**
3. **按如下方式生成有序序列（将 A 置为空）：**
   - For $x = 1$ to U
     - while $B[x] > 0$ do
       - 将整数 x 追加到 A。
       - $B[x] = B[x] - 1$

示例

考虑整数集合： [2,3,8,7,1,2,2,2,7,3,9,8,2,1,4,2,4,6,9,2] 。

一种简单直接的排序方法：

1. **创建计数数组**
2. **统计出现次数**
3. **遍历并输出**



一种简单的排序算法?

该算法称为计数排序。它处理排序问题的一种特例：整数来自较小值域。

两个主要性质：

- **非比较排序：依靠统计元素频次并用数组下标，而非比较。**
- **适用范围有限：仅适用于整数，或可映射为整数的离散数据。**

计数排序的分析

- 第 1 步和第 3 步耗时 $O(U)$。
- 第 2 步耗时 $O(n)$。
- 因此，计数排序的总体运行时间为 $O(n + U) = O(U)$.

- 当 $U = O(n)$ 较小时（例如 $1000n$），计数排序的运行时间为 $O(n)$。

下周课程安排

[Tengfei Chang] 赴欧洲参加 EWSN 会议：
https://ewsn26.netd.cs.tu-dresden.de/

[Zijun Gong] 将于 9 月 15 日授课
（曾于 25-26 学年春季学期讲授 UFUG 2602 ）

[Beiyan Cao] 将于 9 月 17 日授课
（ 26-27 学年秋季学期 UFUG 2602 平行班授课教师）

[Tengfei Chang] 在校授课



September

1			1	2	3	4	5
2	6	7	8	9	10	11	12
3	13	14	15	16	17	18	19
4	20	21	22	23	24	25	26
5	27	28	29	30			

October

					1	2	3
5	4	5	6	7	8	9	10
6	11	12	13	14	15	16	17
7	18	19	20	21	22	23	24
8	25	26	27	28	29	30	31

November

9	1	2	3	4	5	6	7
10	8	9	10	11	12	13	14
11	15	16	17	18	19	20	21
12	22	23	24	25	26	27	28
13	29	30					

December

13			1	2	3	4	5
13	6	7	8	9	10	11	12

随堂测验安排

- 随堂测验的具体安排将于本周公布
  - 可能在 9 月 24 日或 29 日
- 冒泡排序
- 选择排序
- 计数排序

练习 3

- 对以下序列执行**计数排序**：
  - [3, 1, 4, 1, 2, 0, 3]
- **步骤 1 — 创建计数数组**
- **步骤 2 — 重建排序后的数组**

练习 4 - 判断排序算法

- 给定以下执行过程：

[6,5,2,4]

[2,5,6,4]

[2,4,6,5]

[2,4,5,6]

- **问题：**
  该过程是由哪种排序算法产生的?
  - 冒泡排序
  - 选择排序
- **请解释你的答案。**

练习 5 - 哪个序列不可能出现?

- 初始数组:

  [7,4,6,2]

- 下列哪个中间状态在冒泡排序过程中**不可能**出现?

[6,4,2,7]

[2,4,6,7]

[4,6,2,7]

[4,2,6,7]

练习 6 - 修正冒泡排序

- 下面的代码本意是实现冒泡排序，但其中有错误：

```
for i in range(n):
    for j in range(n):
        if A[j] > A[j+1]:
            swap(A[j], A[j+1])
```

问题：

1. A[j+1] 可能会出现什么问题？
2. 内层循环的范围应如何修改？
3. 每一轮之后内层循环能否缩短？
4. 写出修正后的版本。

练习 7

- 对于数组

[1,2,3,4,5]

- 哪种算法可能表现最好?

冒泡排序

选择排序

计数排序

- 假设

冒泡排序使用提前终止标志。

计数排序已知取值范围很小。

测验

- **Q1. 多种简单排序算法都用到了哪种共同思想?**
- A. 将数组随机划分为更小的部分
  B. 逐步构建数组的已排序部分
  C. 总是交换第一个和最后一个元素
  D. 总是使用递归

排序算法比较

排序算法	时间复杂度	空间复杂度	稳定性
插入排序	- 最好： O(n) （已有序） - 最坏： $O(n^2)$	O(1)	稳定
冒泡排序	- 最好： O(n) （已有序） - 最坏： $O(n^2)$	O(1)	稳定
选择排序	- $O(n^2)$	O(1)	不稳定
计数排序	- O(n+U)	O(U)	稳定

若相同值在输出数组中的相对顺序与输入数组相同，则该算法是稳定的。

大纲

- 基本排序
  - 冒泡排序
  - 选择排序
  - 计数排序
- **链表**
  - 单向链表
  - 双向链表
  - 链式序列与数组序列

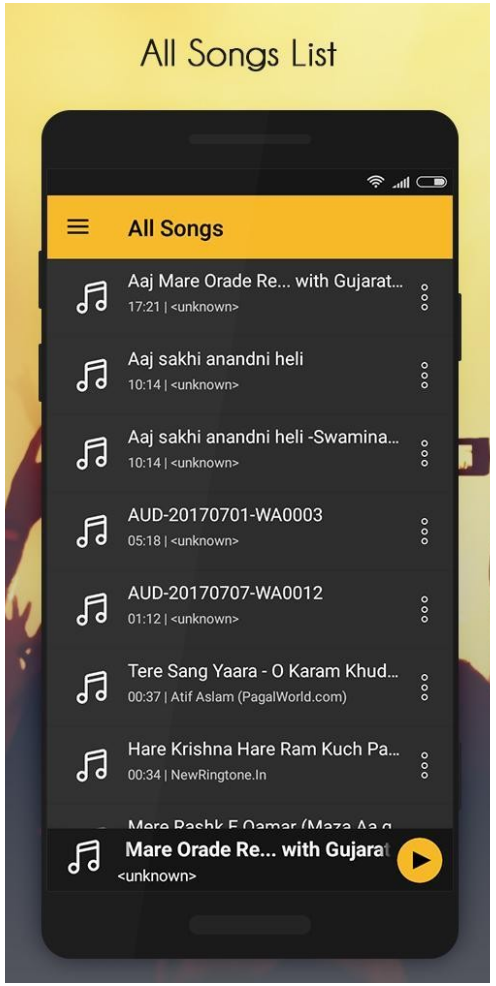
链表– 示例



设想你在开发一款可创建和管理播放列表的音乐播放器。用户经常向播放列表增删歌曲。常见操作包括：

- 在指定位置插入新歌曲
- 从列表中删除已有歌曲

问题： 能否用动态数组管理播放列表？有何优缺点？



引言：数组与链表 (1)

- **基于数组的序列**
  - 连续内存块，存放多个元素。

- **局限性**
  - 动态数组可能有额外未使用空间。
  - 在中间插入或删除元素代价较高。

引言：数组与链表（2）

- **链表：**
  - 数组的替代方案，可避免其局限性。
  - 每个元素用结点存储，通过指向相邻结点的引用维持顺序。
- **局限性：**
  - 按索引访问元素效率低。
  - 仅查看结点本身无法确定其在链表中的位置。

大纲

- 基本排序
  - 冒泡排序
  - 选择排序
  - 计数排序
- 链表
  - **单向链表**
  - 双向链表
  - 链式序列与数组序列

单向链表（0）

```
class ListNode :
  def __init__( self, data ) :
    self.data = data


  a = ListNode( 11 )
  b = ListNode( 52 )
  c = ListNode( 18 )
```

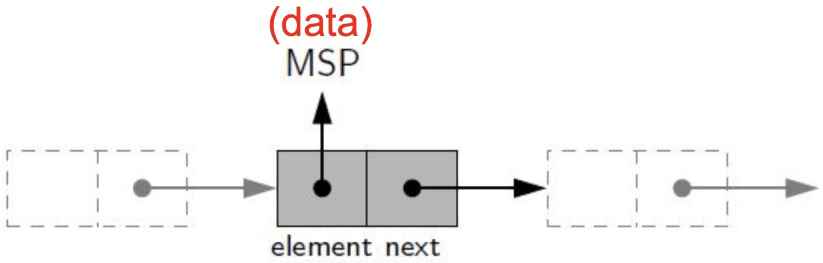
```
class ListNode :
  def __init__( self, data ) :
    self.data = data
    self.next = None

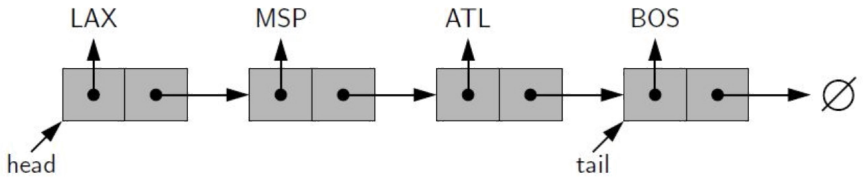
a.next = b

b.next = c
```

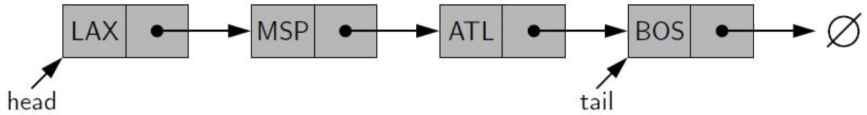
单向链表（1）

- 单向链表是由结点组成的序列
  - **结点内容：每个结点保存一个元素及指向下一结点的引用。**
  - **非连续存储：结点不必位于连续内存地址**
  - **头与尾：第一个结点是头结点；最后一个结点是尾结点。**
  - **遍历：从头结点出发，沿 next 引用到达尾结点，称为链路跳跃或指针跳跃。**





Singly linked list for storing airport codes



Simplified view

单向链表（2）

链表实例至少应保存：

- **头引用：用于定位第一个结点并访问整条链表。**
- **尾引用：可选，便于不经遍历即可快速访问最后一个结点。**
- **大小计数：可选，便于跟踪结点数而无需每次遍历链表。**

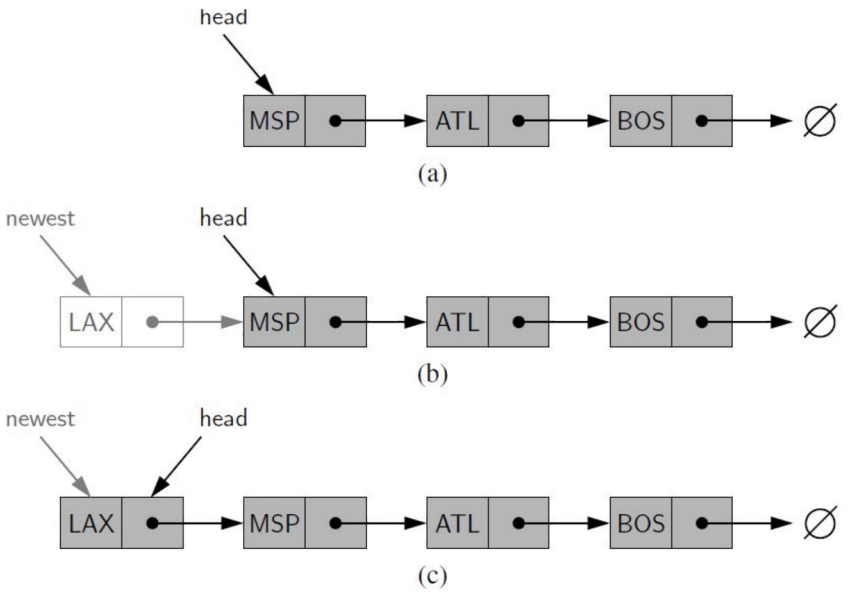
遍历链表并查找结点

```
1  def traversal( head ):
2    curNode = head
3    while curNode is not None :
4      print curNode.data
5      curNode = curNode.next
```

```
1  def unorderedSearch( head, target ):
2    curNode = head
3    while curNode is not None and curNode.data != target :
4      curNode= curNode.next
5    return curNode is not None
```

在单向链表头部插入元素

- 一个重要性质：
  - 空间与当前元素个数成正比。
  - 可以方便地在链表头部插入元素。
- 插入元素的步骤：
  - 创建新结点。
  - 将其元素设为新元素。
  - 将其 next 链接指向当前头结点。
  - 将链表的 head 指向新结点。



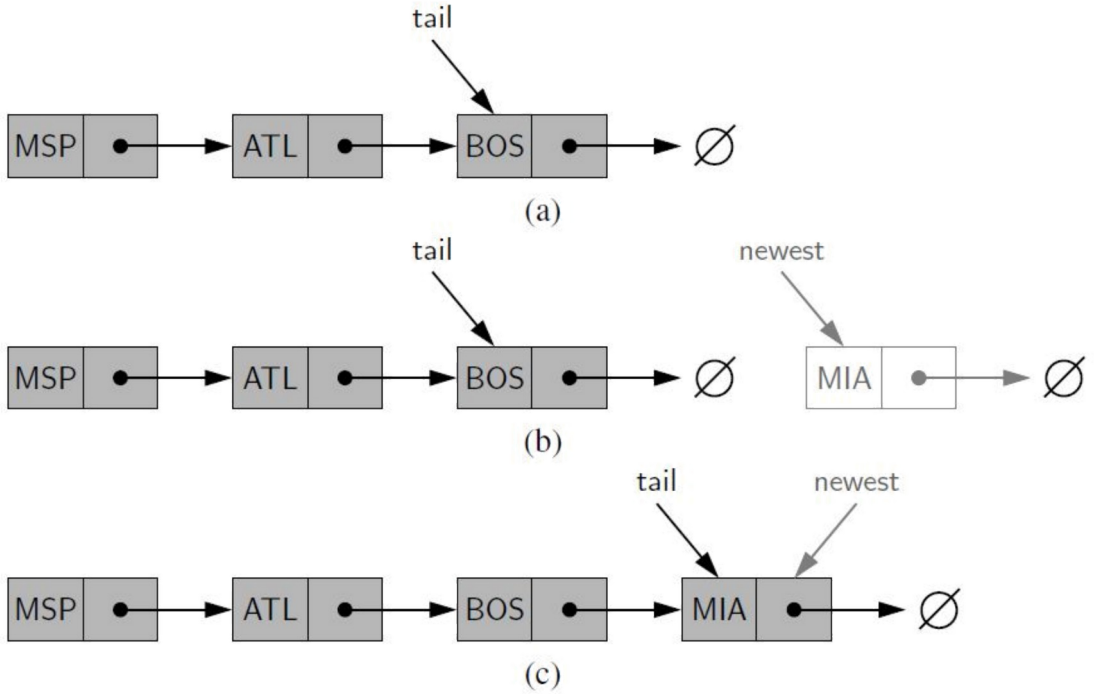
Algorithm add_first(L, e):

```
newest = Node(e)  {create new node instance storing reference to element e}
newest.next = L.head  {set new node's next to reference the old head node}
L.head = newest  {set variable head to reference the new node}
L.size = L.size + 1  {increment the node count}
```

在单向链表尾部插入元素

步骤：

- 创建新结点。
- 将其 next 引用设为 None 。
- 将尾结点的 next 引用指向该新结点。
- 将尾引用更新为该新结点。

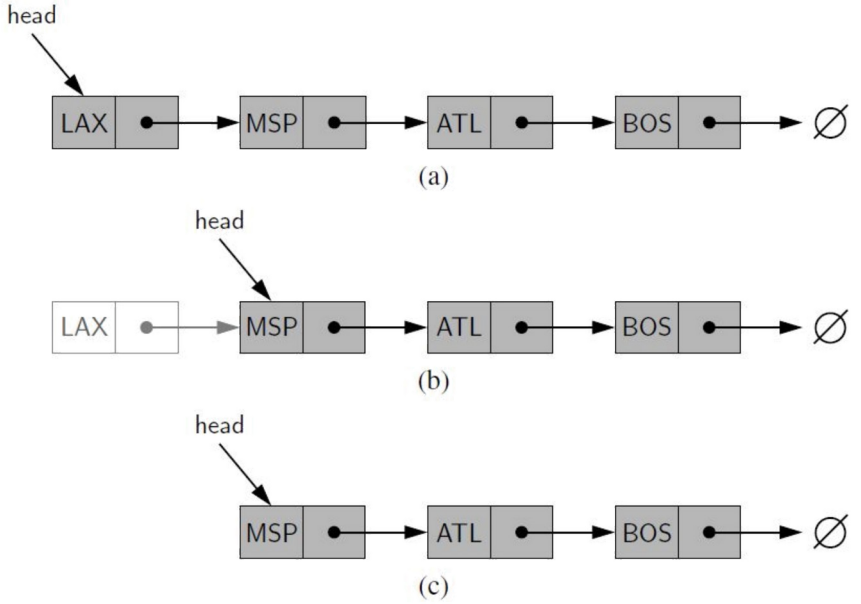


Algorithm add_last(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = None    {set new node's next to reference the None object}
L.tail.next = newest  {make old tail node point to new node}
L.tail = newest       {set variable tail to reference the new node}
L.size = L.size + 1   {increment the node count}
```

从单向链表中删除元素

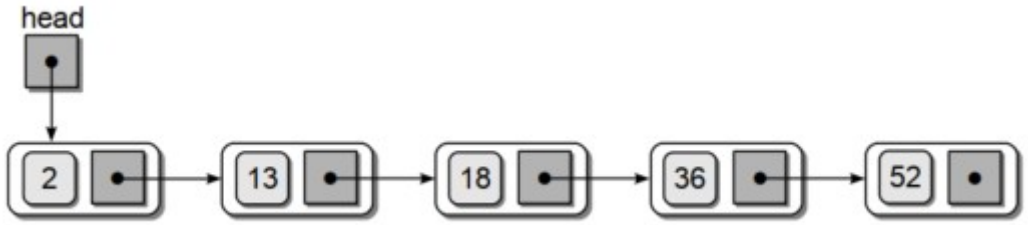
- 从头删除：容易，只需将头部插入过程反过来。
- 删除最后一个结点：较难，因为需要访问倒数第二个结点。
- 访问前驱结点：需要从头开始遍历链表。



Algorithm remove_first(L):
  if L.head is None **then**
    Indicate an error: the list is empty.
  L.head = L.head.next {make head point to next node (or None)}
  L.size = L.size − 1 {decrement the node count}

有序链表

- 链表中的元素可按升序或降序排列



查找有序链表

```
1  def sortedSearch( head, target ) :
2    curNode = head
3    while curNode is not None and curNode.data <= target :
4      if curNode.data == target :
5        return True
6      else :
7        curNode = curNode.next
8    return False
```

向有序链表插入值

```
1  # Given the head pointer, insert a value into a sorted linked list.
2   # Find the insertion point for the new value.
3  predNode = None
4  curNode = head
5  while curNode is not None and value > curNode.data :
6    predNode = curNode
7    curNode = curNode.next
8
9   # Create the new node for the new value.
10 newNode = ListNode( value )
11 newNode.next = curNode
12  # Link the new node into the list.
13 if curNode is head :
14   head = newNode
15 else :
16   predNode.next = newNode
```

单向链表练习 1

根据下面的代码：
- 画出内存结构（方框 + 箭头）。
- 标注 **head**、**a**、**b**、**c**。

```
class Node:
    def __init__(self, v):
        self.val = v
        self.next = None

a = Node(7)
b = Node(13)
c = Node(5)

a.next = c
c.next = b
head = a
```

单向链表练习 2

- 假设有一个长度为 n 的单向链表

操作	时间复杂度
头部插入	?
尾部插入（无尾引用）	?
尾部插入（有尾引用）	?
按值查找	?
访问下标 i	?
在已知结点之后插入	?

单向链表练习 3

- 给定一个结点引用 cur，它指向单向链表中间的某个结点。

任务：用一个新结点在 cur 之后立即插入值 x，补全缺失的代码行：

```
new = Node(x)

_____________________

_____________________
```

问题：下面的顺序会出现什么问题?

```
cur.next = new
new.next = cur.next
```

单向链表练习 4

- 给定

```
head = head.next
```

- 问题：

1. 执行这条语句后， head 指向哪里?
2. 原来的第一个结点是否仍是链表的一部分?
3. 程序还能通过 head 访问到原来的第一个结点吗?
4. 在 Python 中，它最终会怎样?

单向链表练习 5

- 假设给定单向链表中结点 x 的引用。
- 你**没有**头结点引用。
- **问题：**能否将 x 从逻辑链表中删除?
- 考虑两种情况：

1. x **不是尾结点**
2. x **是尾结点**

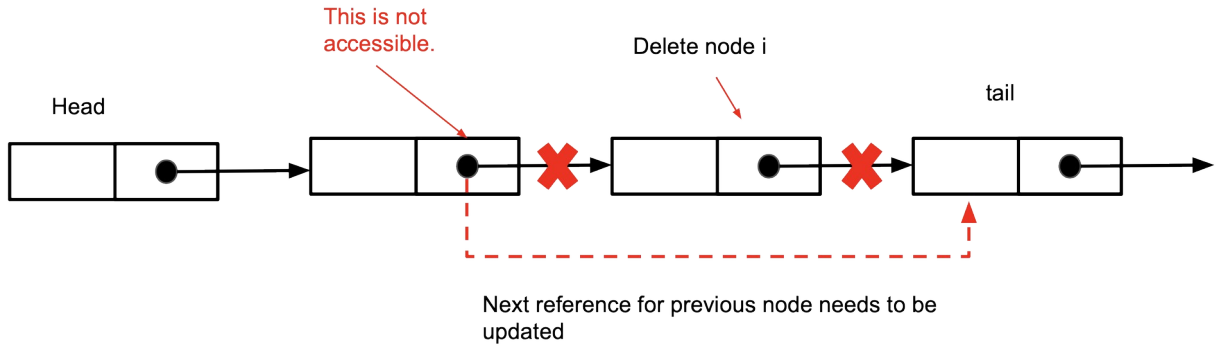
大纲

- 基本排序
  - 冒泡排序
  - 选择排序
  - 计数排序
- 链表
  - 单向链表
  - **双向链表**
  - 链式序列与数组序列

双向链表——动机

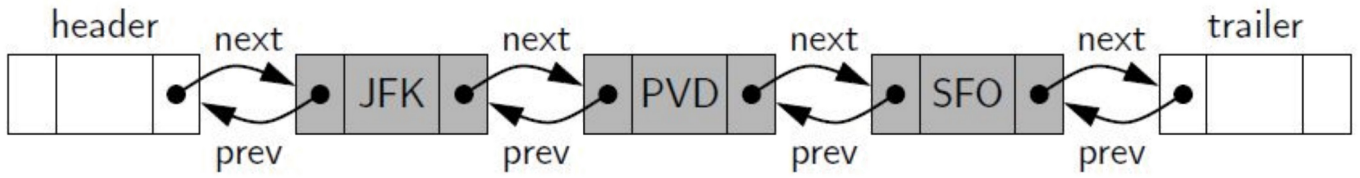
- **单向链表的局限：每个结点只指向下一个。**

- 难点：
  - 难以高效删除最后一个结点。
  - 更一般地，若只有某结点的引用，则难以删除该结点，因为不易找到前驱以更新其链接。



双向链表（1）

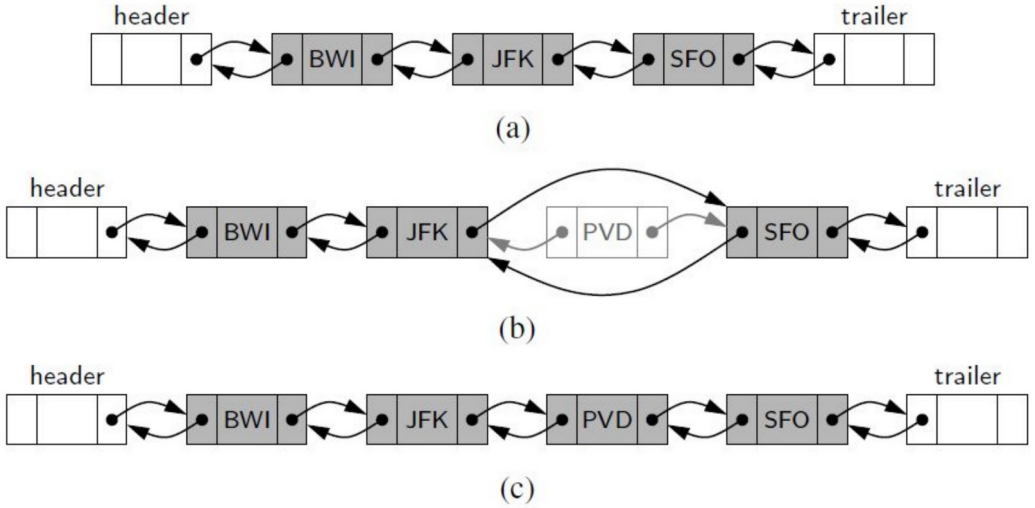
- **结构：每个结点同时链接后继与前驱。**
- **优点：**
  - 可在链表任意位置快速插入与删除（O(1) 时间）。
- **哨兵结点：**
  - 在首尾使用特殊的 "header" 与 "trailer" 结点。
  - 这些哑结点使边界情况更易处理。
  - 空链表中， header 指向 trailer ， trailer 回指 header 。



双向链表（2）

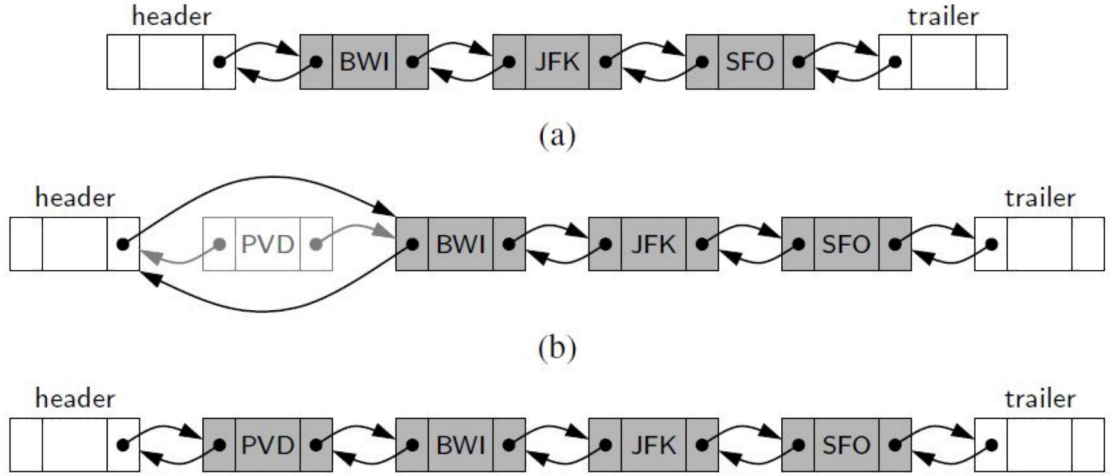
- **哨兵结点：**
  - 保持 header 与 trailer 固定，从而简化操作。
- **统一操作：**
  - 将插入与删除视为已有结点之间的链接改动。

用双向链表插入

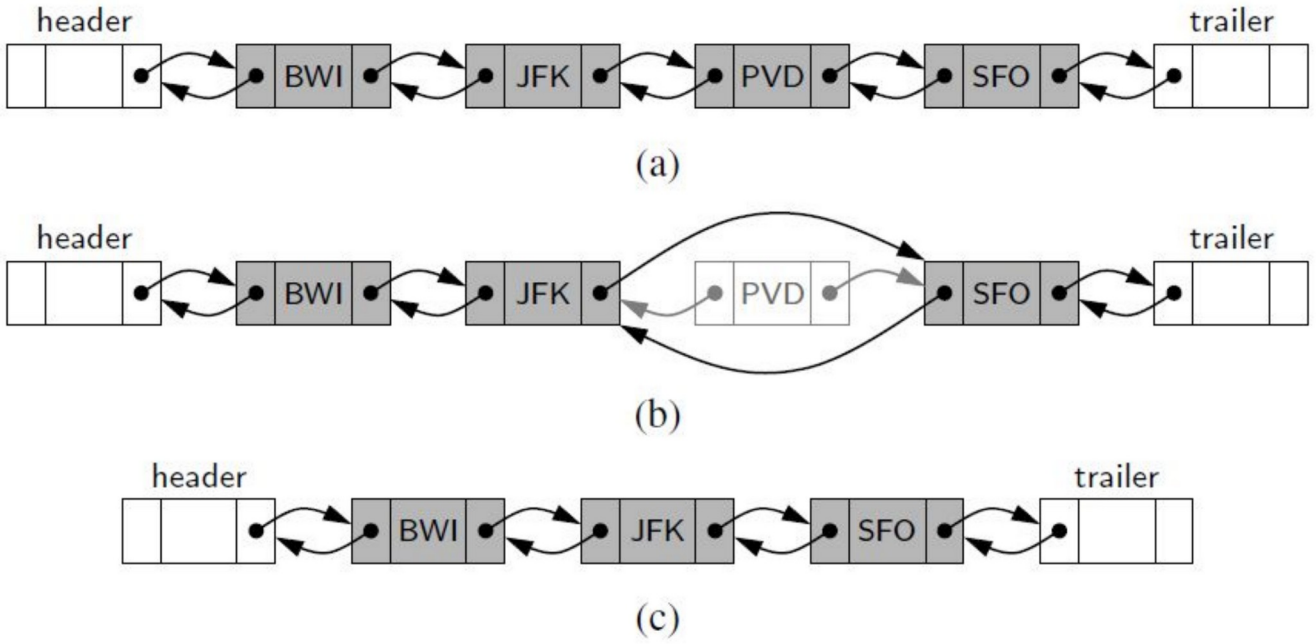


情形 I： 在非空链表的任意位置插入。

情形 II： 在链表开头插入



在双向链表中删除结点



删除任意位置的结点

由于使用哨兵，删除首结点或尾结点可用同一实现。

双向链表的基本实现（1）

```
1 class _DoublyLinkedBase:
2   '''A base class providing a doubly linked list representation.'''
3
4   #-------------------------------------------------------------------
5   class _Node:
6     '''Lightweight, nonpublic class for storing a doubly linked node.'''
7     __slots__ = '_element', '_prev', '_next'         # streamline memory
8
9     def __init__(self, element, prev, next): # initialize node's field
10      self._element = element       # element to be stored
11      self._prev = prev             # Previous node reference
12      self._next = next             # next node reference
13  #-------------------------------------------------------------------
14  def __init__(self):
15    '''Create an empty list.'''
16    self._header = self._Node(None, None, None)
17    self._trailer = self._Node(None, None, None)
18    self._header._next = self._trailer      # trailer is after header
19    self._trailer._prev = self._header      # header is before trailer
20    self._size = 0                          # Number of elements
21
22  def __len__(self):
23    '''Return the number of elements in the list.'''
24    return self._size
25
26  def is_empty(self):
27    '''Return True if list is empty.'''
28    return self._size == 0
```

Base Class declaration

We maintain two references: **`_prev`** & **`_next`**

双向链表的基本实现（1）

```
30  def _insert_between(self, e, predecessor, successor):
31    '''Add element e between two existing nodes and return new node.'''
32    newest = self._Node(e, predecessor, successor) # linked to neighbors
33    predecessor._next = newest
34    successor._prev = newest
35    self._size += 1
36    return newest
37
38  def _delete_node(self, node):
39    '''Delete nonsentinel node from the list and return its element.'''
40    predecessor = node._prev
41    successor = node._next
42    predecessor._next = successor
43    successor. prev = predecessor
44    self._size -= 1                  # record deleted element
45    element = node._element
46    node._prev = node._next = node._element = None # deprecate node
47    return element      # return deleted element
```

双向链表练习 1

- 假设链表长度为 n。补全下表：

操作	单向链表	双向链表
正向遍历	?	?
反向遍历	?	?
给定非头结点引用，删除该结点	?	?
每个结点额外的指针内存	?	?

双向链表练习 2

- 补全缺失的指针
  - 初始链表：

    H <-> P <-> R <-> T
  - 将结点 Q 插入到 **P 和 R 之间**

Q.prev = __
Q.next = __
__.next = __
__.prev = __

双向链表练习 3

- 已知：

```
A <-> C
```

- 要将 B 插入它们之间。考虑如下代码：

```
A.next = B
B.prev = A
B.next = C
C.prev = B
```

问题
最终结构是否正确?
在更新过程中，是否有某个时刻丢失了从 A 到 C 的原有连接?
你能写出一个先保存原相邻结点的更安全版本吗?

双向链表练习 4

- 链表

  H <-> A <-> B <-> C <-> T

- 给定变量 b，引用结点 B。

任务：填写删除 B 所需的两条关键指针更新：

b.prev.next = _____

b.next.prev = _____

双向链表练习 5

- 给定

```
... <-> A <-> B <-> ...
```

- 给定 A 与 B 的引用，已知二者相邻且 A.next == B。
- **任务:**
- 令 P = A.prev， N = B.next
- 交换二者，使其变为:

```
... <-> B <-> A <-> ...
```

填空:

```
B.prev = _____
B.next = _____

A.prev = _____
A.next = _____

if P is not None:
    P.next = _____

if N is not None:
    N.prev = _____
```

双向链表练习 6

- 两个链表（均含哨兵）：
- List1：`H1 <-> A <-> B <-> T1`

- List2：`H2 <-> X <-> Y <-> T2`

- **任务：**将整个 List2 （X..Y）拼接到 List1 中**结点 A 之后**，得到：

```
H1 <-> A <-> X <-> Y <-> B <-> T1
```

假设已有引用： A、 B （A.next）、 X （H2.next）、 Y （T2.prev）。

链式序列与数组序列对比

特性	数组	链表
访问时间	按下标访问元素为 O(1)	已知节点引用时为 O(1)
插入 / 删除	O(n) ，需移动元素	任意位置均为 O(1)
存储	连续	由小块组成，不必连续
内存占用	O(n) 空间；更省内存，只存储元素	O(n) 空间；较费内存，需存储元素和指针
性能一致性	因扩容可能导致性能波动	性能稳定，时间界可预测
适用场景效率	更适合频繁访问的场景	更适合频繁插入 / 删除的场景